

Asignatura: Desarrollo de Aplicaciones Web (IS093A)

Unidad I: Desarrollo Web Frontend

Tema: Framework JS: Eventos, Renderizado Condicional/Iterativo, Formularios, Routing y Consumo de APIs (Async/Await, Axios)

Modalidad: Presencial / Laboratorio

Duración: 90 minutos

APELLIDOS Y NOMBRES: Alarcón Mendoza Estiven Rodrigo

Paso 1: Creación del Proyecto

```
PS D:\IX SEMESTRE\Desarrollo Web\Semana 6 - Miercoles\Framework JS> npm install

up to date, audited 137 packages in 905ms

31 packages are looking for funding
  run `npm fund` for details

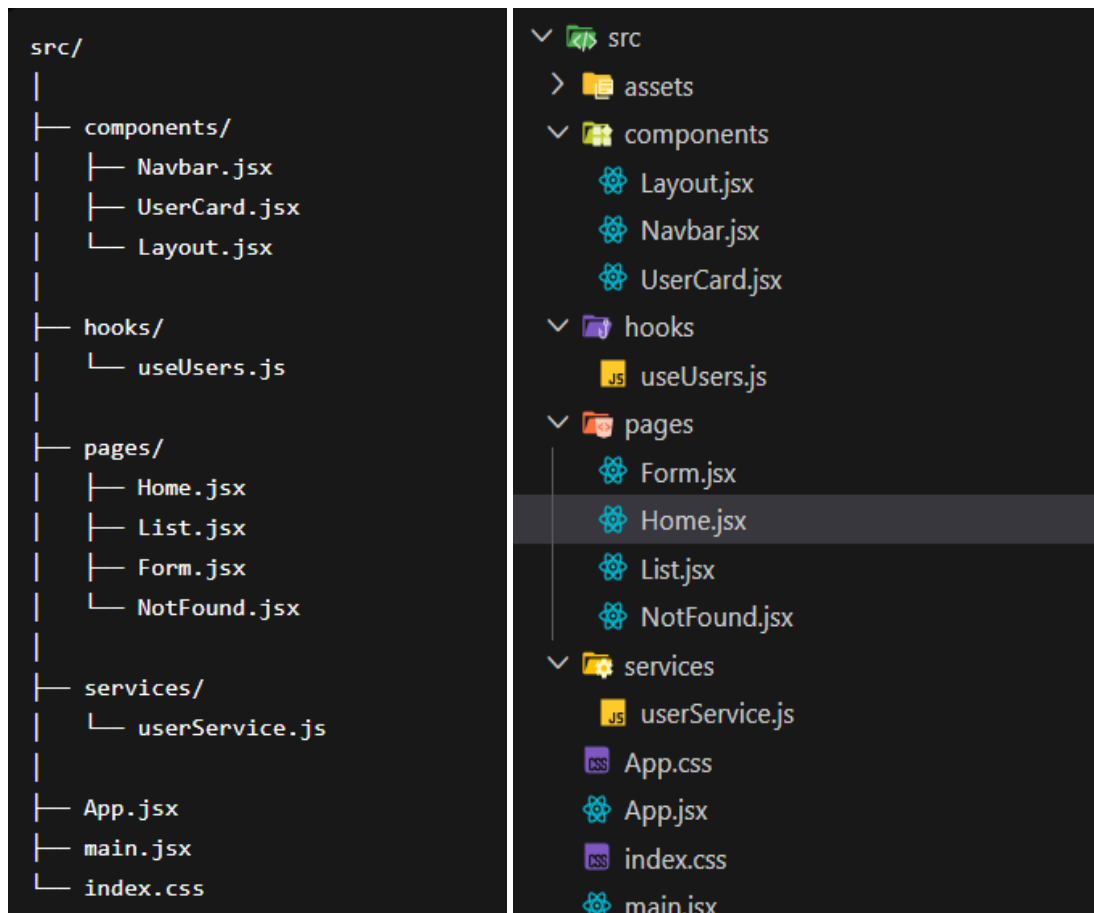
found 0 vulnerabilities
PS D:\IX SEMESTRE\Desarrollo Web\Semana 6 - Miercoles\Framework JS> npm install react-router-dom axios

added 29 packages, and audited 166 packages in 4s

38 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Paso 2: Estructura del proyecto



Paso 3: Consumo de API con Axios y Async/Await

Se creó un servicio llamado `userService.js`, encargado de realizar la petición HTTP a la API:

<https://jsonplaceholder.typicode.com/users>

También se creó el hook personalizado `useUsers.js`, donde se manejan los estados:

```
loading
data
error
```

La petición fue realizada con `async/await` dentro de `useEffect`. Además, se utilizó `AbortController` para cancelar la petición si el componente se desmonta antes de que finalice la carga de datos.

```
import axios from "axios";

const API_URL = "https://jsonplaceholder.typicode.com/users";

export async function getUsers(signal) {
  const response = await axios.get(API_URL, { signal });
  return response.data;
}
```

```
import { Link } from "react-router-dom";

function NotFound() {
  return (
    <section className="page">
      <h1>Error 404</h1>
      <p>La página solicitada no existe.</p>
      <Link to="/" className="btn-link">Volver al inicio</Link>
    </section>
  );
}

export default NotFound;
```

Paso 4: Renderizado iterativo, condicional y formulario controlado

En la página `List.jsx` se aplicó renderizado iterativo mediante `.map()` para mostrar los usuarios obtenidos desde la API. Cada elemento utiliza una clave única y estable mediante `user.id`.

También se aplicó renderizado condicional para mostrar mensajes según el estado de la aplicación:

```
loading && ...
error && ...
```

```
users.length === 0 && ...  
users.length > 0 && ...
```

En la página Form.jsx se implementó un formulario controlado con useState, donde cada campo está vinculado al estado del componente. Se manejaron los eventos onChange y onSubmit.

Paso 5: Integración y validación

Se integraron los componentes principales de la aplicación: Navbar, Layout, UserCard, páginas y servicios. El flujo de datos se organizó separando responsabilidades:

```
services/ → consumo de API  
hooks/    → lógica reutilizable  
pages/    → vistas principales  
components/ → componentes reutilizables
```

```
import { NavLink } from "react-router-dom";  
  
function Navbar() {  
  return (  
    <header className="navbar">  
      <h2>Práctica React SPA</h2>  
  
      <nav>  
        <NavLink to="/" className={({ isActive }) => isActive ? "active"  
: ""}>  
          Inicio  
        </NavLink>  
  
        <NavLink to="/usuarios" className={({ isActive }) => isActive ?  
"active" : ""}>  
          Usuarios  
        </NavLink>  
  
        <NavLink to="/formulario" className={({ isActive }) => isActive ?  
"active" : ""}>  
          Formulario  
        </NavLink>  
      </nav>  
    </header>  
  );  
}  
  
export default Navbar;
```

```
import Navbar from "../Navbar";  
  
function Layout({ children }) {
```

```

return (
  <>
    <Navbar />
    <main className="container">
      {children}
    </main>
  </>
);
}

export default Layout;

```

```

function UserCard({ user }) {
  return (
    <article className="card">
      <h3>{user.name}</h3>
      <p><strong>Usuario:</strong> {user.username}</p>
      <p><strong>Email:</strong> {user.email}</p>
      <p><strong>Ciudad:</strong> {user.address.city}</p>
      <p><strong>Empresa:</strong> {user.company.name}</p>
    </article>
  );
}

export default UserCard;

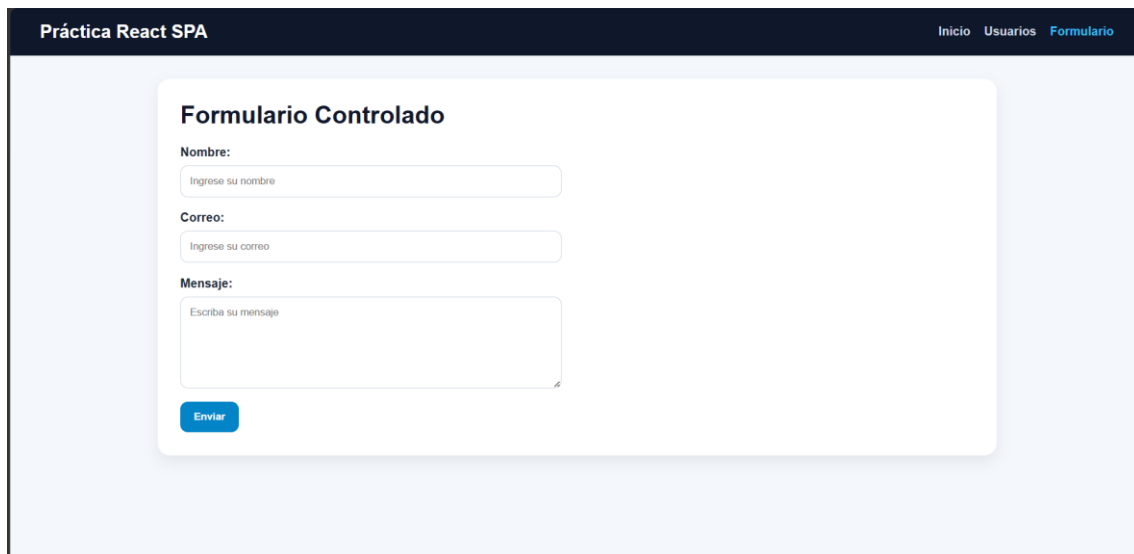
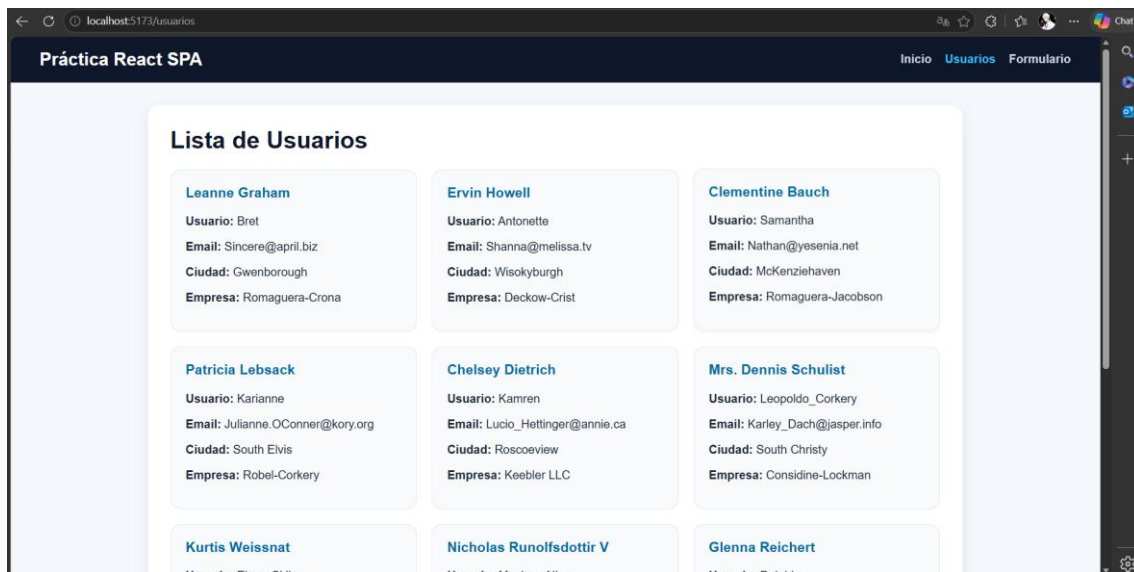
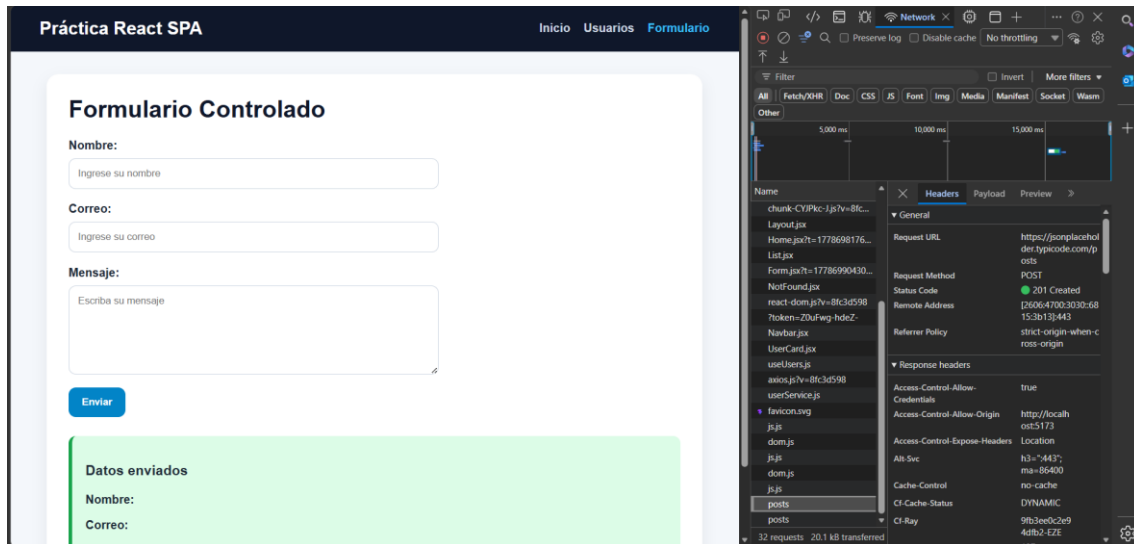
```

El renderizado innecesario se reduce porque la petición a la API se ejecuta solo una vez al cargar el componente, gracias al arreglo vacío [] en useEffect.

Resultado:

The screenshot shows a web application titled "Práctica React SPA" with a navigation bar containing "Inicio", "Usuarios", and "Formulario". The main content area displays a "Lista de Usuarios" with a grid of user cards. Each card shows a user's name, username, email, city, and company. The network inspector on the right shows a GET request to "https://jsonplaceholder.typicode.com/users" with a status code of 200 OK. The response headers include "Access-Control-Allow-Origin" and "Cache-Control".

Nombre	Usuario	Email	Ciudad	Empresa
Leanne Graham	Bret	Sincere@april.biz	Gwenborough	Romaguera-Crona
Ervin Howell	Antonette	Shanna@melissa.tv	Wisokyburgh	Deckow-Crist
Clementine Bauch	Samantha	Nathan@yesenia.net	McKenziehaven	Romaguera-Jacobson
Patricia Lebsack	Karianne	Julianne.OConner@kory.org	South Elvis	Robel-Corkery
Chelsey Dietrich	Kamren	Lucio_Hettinger@annie.ca	Roscoeview	Keebler LLC
Mrs. Dennis Schulist	Leopoldo_Corkery	Karley_Dach@jasper.info	South Christy	Considine-Lockman
Kurtis Weissnat				
Nicholas Runoldsson				
Glenna Reichert				



Link de GitHub: [Ichimaru465/Framework-JS-S6-DW](https://github.com/Ichimaru465/Framework-JS-S6-DW)